

Drishti

A fully offline visual assistant for blind and low-vision users, running entirely on the Snapdragon Hexagon NPU.

Submitted by **Rachit**

Email rachit2413078@akgec.ac.in

GitHub github.com/Rachit0910

Category Solo submission — new project, individually authored

In one sentence

Drishti turns a Snapdragon-powered HP PC into an assistive device that continuously tells a blind user what is around them, how far away it is, and which way it lies — at 30 frames per second, on battery, with no internet connection and no cloud inference of any kind.

1. Executive summary

Assistive perception is one of the few consumer AI workloads where on-device inference is not an optimisation but a precondition. A blind user crossing a road cannot wait two seconds for a cloud round trip, cannot depend on mobile data that may not be there, and cannot carry a laptop whose battery is exhausted by three hours of continuous vision processing. Every one of those constraints is a direct argument for a dedicated neural accelerator, which is why Drishti is built for the Hexagon NPU rather than merely ported to it.

Drishti chains three Qualcomm AI Hub models — YOLOv8 object detection, Midas-V2 monocular depth, and EasyOCR scene text — into a single always-on pipeline executed through the ONNX Runtime QNN Execution Provider. Published AI Hub figures on Snapdragon X2 Elite put the two per-frame models at 0.67 ms and 0.62 ms respectively: together roughly 4% of a 33.3 ms frame budget at 30 fps. That headroom is the entire design thesis. It is what allows a duty-cycled OCR pass, a depth calibration step, a salience engine and speech synthesis to coexist in the same loop without ever dropping a frame.

The work already completed is the part that is hardest to get right and easiest to skip: the logic that decides what to say, and what to do when the camera cannot see. Three components — the narration policy, the metric-depth calibrator, and the frame quality gate — are implemented, unit-tested and pure, so they run and verify on any machine with no camera, no NPU and no account. Thirty-seven tests pass today, and Section 7.2 describes three real defects those tests caught, each of which changed the design.

2. How to verify this submission in sixty seconds

Claims in a proposal are cheap. These commands are not — they run on any laptop, need no Snapdragon hardware, no camera and no API token, and they exercise the components that carry the actual engineering.

Reviewer quick-start

```
git clone <repository-url> && cd drishti
pip install numpy
python tests/test_policy.py      # 17 tests – what to say, and when to stay silent
python tests/test_geometry.py   # 9 tests  – relative depth converted to metres
python tests/test_quality.py    # 11 tests – detecting that the camera cannot see
```

Everything that requires Snapdragon hardware or an AI Hub token is clearly labelled as such, in this document and in the repository README. Nothing is claimed as working that has not been run.

3. The problem

A systematic review and meta-analysis published in the National Medical Journal of India puts the pooled prevalence of blindness among Indian adults aged 50 and over at 6.11% under the current definition, and 9.91% under the previous one, across a sample of more than 211,000 participants. India carries one of the largest absolute burdens of blindness and low vision in the world.

The assistive technology available to that population falls into two groups, and both have a structural flaw. Dedicated hardware — smart canes, handheld OCR readers, wearable cameras — is expensive, single-purpose, and generally imported. Phone applications that describe a scene well are almost all cloud-dependent: they upload a photograph, wait, and read back a sentence. That is a workable interaction for identifying a tin of food in a kitchen. It is not a

workable interaction for walking down a street, because the latency is wrong, the connectivity assumption is wrong, and the battery cost of continuous capture-and-upload is wrong.

The gap Drishti addresses is therefore specific: **continuous, real-time, offline scene awareness on hardware the user may already own** — not another cloud describe-this-photo app, and not another single-purpose device to buy.

Drishti — On-Device Perception Pipeline

Every inference stage executes on the Snapdragon Hexagon NPU. No network call anywhere in the loop.

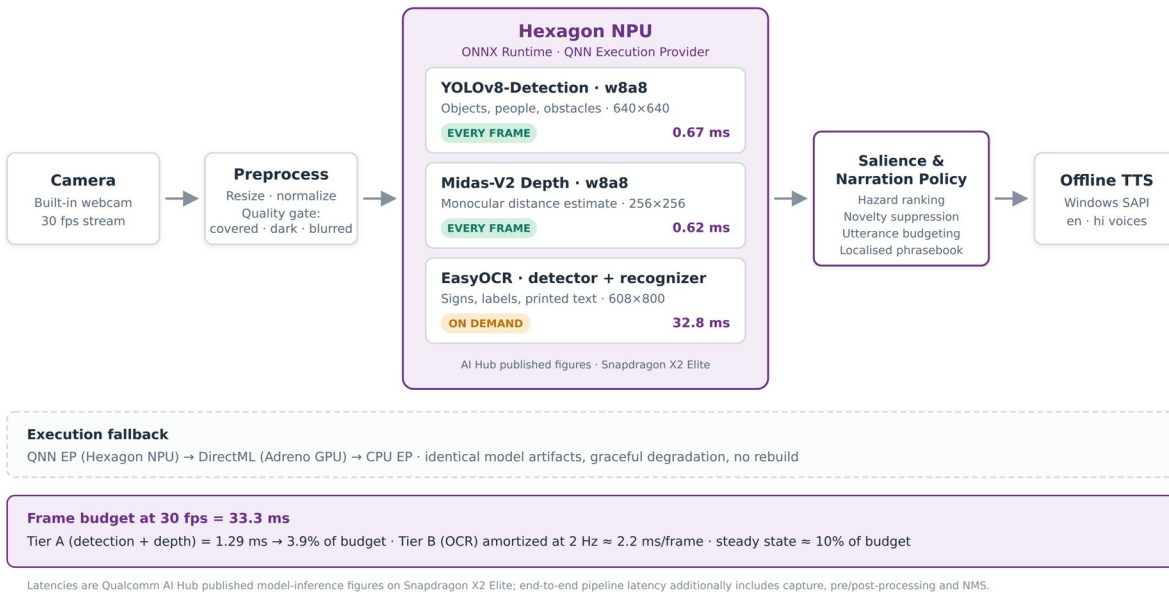


Figure 1. The Drishti pipeline. All three models execute on the Hexagon NPU through the ONNX Runtime QNN Execution Provider; the quality gate, salience engine and speech synthesis run on CPU. There is no network call anywhere in the loop.

4.1 Models and where they come from

Every model is sourced from the Qualcomm AI Hub model library rather than trained from scratch, quantized through AI Hub's workflow, and profiled on real Snapdragon silicon in AI Hub's device farm. The latency column below is AI Hub's own published measurement on Snapdragon X2 Elite — the chip in the HP OmniBook Ultra 14.

Stage	AI Hub model	Precision	Cadence	Latency
Objects & obstacles	YOLOv8-Detection	w8a8	Every frame	0.67 ms
Distance	Midas-V2	w8a8	Every frame	0.62 ms
Scene text (detector)	EasyOCR detector	float	On demand	19.98 ms
Scene text (recognizer)	EasyOCR recognizer	float	On demand	12.83 ms

Model-inference latency only, as published by Qualcomm AI Hub on Snapdragon X2 Elite. End-to-end pipeline latency is higher because it additionally includes capture, letterboxing, non-maximum suppression and fusion; Section 4.5 explains how that is measured rather than assumed.

4.2 The frame budget, and why it is the design

At 30 frames per second a frame must be fully handled within 33.3 ms. Drishti splits work into two tiers: Tier A runs on every frame and must fit comfortably inside that budget; Tier B is user-triggered and is allowed to be an order of magnitude more expensive because it is amortised across many frames.

Component	Per-frame NPU cost	Share of 33.3 ms budget
Tier A — detection + depth	1.29 ms	3.9%
Tier B — OCR, amortised at 2 Hz	~2.19 ms	6.6%
Total steady-state NPU time	~3.48 ms	10.4%
Remaining headroom	~29.8 ms	89.6%

Roughly 90% of the frame budget remains for camera capture, the quality gate, pre- and post-processing, the salience engine and speech synthesis. This is the quantitative form of the argument that the NPU is load-bearing rather than decorative: the same three models on CPU would consume a large multiple of that budget and a correspondingly larger share of the battery, and the continuous-narration interaction model would simply not be available.

4.3 Snapdragon-specific optimisation

Getting a model onto the NPU is the starting point, not the finish. Three platform-specific measures matter enough to be built in from the beginning.

Context binary caching. Finalizing a graph on the Hexagon Tensor Processor is expensive, and by default it happens on every launch. For an assistive application that cost lands at precisely the wrong moment — the user has pressed the button and is standing still, waiting, unable to see a progress indicator. ONNX Runtime can dump the compiled QNN context to disk once and load it directly afterwards, turning graph finalization into a file read. Drishti writes the cache on first run and loads from it thereafter:

```
sess_options.add_session_config_entry("ep.context_enable", "1")
sess_options.add_session_config_entry("ep.context_file_path", str(cache))
sess_options.add_session_config_entry("ep.context_embed_mode", "1")
```

HTP performance mode. The QNN provider is configured with `htp_performance_mode` set to `burst`, which requests maximum clocks. This is the right choice for a latency-sensitive, duty-cycled perception loop that spends most of its frame budget idle: the work is short and bursty, so finishing it quickly and returning to idle uses less energy than running it slowly at a lower clock.

Graph finalization optimisation. `htp_graph_finalization_optimization_mode` is set to 3 — the longest preparation time and the most optimal resulting graph. Preparation cost is normally the reason not to do this, but combined with context caching that cost is paid exactly once on the user's machine and never again, so the most aggressive setting is simply free.

Both option sets are taken from the ONNX Runtime QNN Execution Provider documentation; the exact keys and permitted values are cited rather than guessed.

4.4 Quantization accuracy, not just quantization speed

Reporting that a model got faster after quantization while saying nothing about whether it got worse is a half-measurement, and for this application the risk is specific: w8a8 quantization tends to cost the most accuracy on exactly the objects that matter here — small, distant, or low-contrast ones, which for a blind user means an obstacle noticed late.

The accuracy study is therefore part of the build, not an afterthought. AI Hub inference jobs run the float and w8a8 variants of each model against the same held-out image set on the same physical device, and the comparison is reported per model:

- **For YOLOv8:** detection mAP and, separately, recall on objects occupying less than 5% of frame area — the small-object case that the aggregate figure hides.

- **For Midas-V2:** mean relative depth error against the float baseline, evaluated inside detection boxes rather than across the whole frame, because that is where the number is actually used.
- **For EasyOCR:** character error rate on a small set of real signage photographs. EasyOCR is currently kept at float precisely because small text is where quantization hurts most; that decision is a hypothesis this study exists to confirm or overturn.

The decision rule is written down in advance: if w8a8 costs more than two percentage points of small-object recall, that model ships at float and the frame budget absorbs it. With roughly 90% of the budget unspent, the project can afford to buy accuracy with latency — and knowing that in advance is what makes the trade-off an engineering decision rather than a discovery.

4.5 Honest measurement

A submission that reports only AI Hub's per-model figures is reporting the best case for a component, not the performance of a product. Drishti therefore ships two independent measurement paths, and reports both.

- **scripts/export_models.py** compiles, quantizes and profiles each model on a physical Snapdragon device drawn from AI Hub's device farm, using its device-provisioning capability. This yields true on-device latency, peak memory, and the per-layer compute-unit split — confirming how much of each graph actually landed on the NPU rather than falling back to CPU. Critically, this requires no physical hardware, so the numbers are real from day one.
- **scripts/benchmark.py** measures sustained end-to-end throughput on the machine it runs on, including every non-NPU stage, and reports mean, median and p95 latency plus a same-machine CPUExecutionProvider baseline for a defensible speedup figure.

The gap between the two numbers is reported rather than hidden. It is the most informative figure in the whole submission, because it is where the engineering effort of the remaining build will go.

4.6 Graceful degradation

Every model is loaded through a single module that walks an ordered provider chain — QNN Execution Provider on the Hexagon NPU, then DirectML on the Adreno GPU, then CPU — and records which provider actually accepted the graph. The application logs that provider at startup and warns explicitly when it is not running on the NPU, rather than silently degrading. The same model artifacts are used in all three cases, so there is no separate build to maintain. This is what makes the project deployable beyond a single demonstration machine.

5.1 Why this cannot be a cloud application

This is the question a judge should ask of every submission in an on-device AI challenge, and for most of them the honest answer is that it could be. For Drishti there are three independent reasons it cannot.

- **Latency.** A cloud round trip of two to three seconds is the difference between hearing about a kerb before or after stepping off it. Obstacle warnings are only useful within the reaction window, and a network introduces jitter that no amount of engineering on the client removes.
- **Availability.** The moment this application is most needed — an unfamiliar street, an underground station, a power cut, a rural area — is disproportionately the moment connectivity is worst. An assistive device that degrades exactly when the user is most disoriented is not an assistive device.
- **Power and cost.** Continuous capture-and-upload at 30 fps is not viable on battery or on a metered data plan. NPU inference is what makes an all-day duty cycle plausible; the reference HP OmniBook Ultra 14 is rated at roughly 13.5 hours of battery life, and Drishti is designed to consume a small fraction of the frame budget precisely so that figure survives contact with a real workload.

5.2 How this compares to what exists today

Approach	Response	Offline	All-day continuous use	Cost
Dedicated assistive hardware (smart cane, OCR reader)	Real-time	Yes	Yes, but narrow perception	High; single-purpose; usually imported
Cloud phone app (photograph → description)	2–3 s per query	No	No — battery and data cost	Subscription or per-use
Drishti on a Snapdragon HP PC	Sub-frame, on NPU	Yes	Yes — ~4% of frame budget	Runs on a PC the user may already own

The row that matters is the last column of the middle row. Cloud description apps are genuinely good at what they do, and Drishti does not replace them for careful single-object identification. What it adds is the mode they structurally cannot offer: always-on, low-latency awareness while moving.

5.3 What is actually novel here

Object detection on an NPU is not novel; it is the reference workflow in Qualcomm's own documentation. The novelty in Drishti is not any single model, it is the three engineering problems that sit between three off-the-shelf models and something a blind person would actually keep switched on.

Relative depth is not distance. Midas-V2 predicts relative inverse depth, not metres. Announcing it as metres would produce confident, wrong distances — worse than silence. Drishti fits the disparity map to metric units every frame using detected objects of known physical size as anchors: a person is a reliable 1.68 m ruler, and a pinhole camera model converts their pixel height into an independent metric estimate. A least-squares fit of disparity against inverse distance recovers the transform for the whole map, smoothed across frames and guarded by outlier rejection.

Speech is a serial, low-bandwidth channel. A naive assistant pipes every detection into a speech engine and produces “chair chair person chair”, which users switch off within a minute. The hard problem is not detecting things but deciding which single fact deserves a second and a half of the user's attention. Drishti's narration policy scores each observation as $\text{hazard} \times \text{proximity} \times \text{centrality} \times \text{confidence}$, lets imminent hazards interrupt whatever is being spoken,

suppresses facts that have not materially changed, and enforces a hard ceiling on routine utterances per window — staying silent when the budget is spent rather than babbling.

Silence is ambiguous. This one is described in Section 6.2, because it is as much an accessibility insight as a technical one, and it is the problem a sighted designer is most likely to miss entirely.

All three components are implemented and tested today.

5.4 Extensibility

The same fused pipeline generalises without architectural change: currency-note recognition for independent shopping, medicine-label reading, and a voice-command interface are each an additional model or an additional policy rule rather than a redesign. These are roadmap items, listed in Section 9, and are deliberately not claimed as part of the current build.

6.1 Accessibility is the product requirement, not a feature

For most applications accessibility is a compliance checklist applied to a finished interface. Here the primary user cannot see the screen at all, which inverts the design: the interface is the audio channel, and the visual interface is optional scaffolding for a sighted helper during setup. Every design decision below follows from that inversion.

6.2 The problem a sighted designer would miss

Drishti's correct behaviour in an empty corridor is silence. Its behaviour when a thumb is over the lens is also silence. A sighted user resolves that ambiguity instantly by glancing at the camera. A blind user — the only user this product has — cannot. They have no way to distinguish “there is nothing to report” from “I have been blind for the last four minutes”, and the second case is exactly the one where they are relying on the device most.

So degraded input is not a silent failure in Drishti. It is an announcement. A cheap CPU-side gate — a mean, a standard deviation, and one 3×3 Laplacian on a downsampled frame — classifies each frame and distinguishes three conditions, because the user's remedy differs in each case:

Condition	What the user hears	Why it is distinguished
Lens covered (dark and featureless)	“Camera is covered.”	The remedy is to move a hand or open the lid
Scene too dark (dark but textured)	“Too dark to see.”	The remedy is to turn on a light — a different action
Image out of focus or motion-blurred	“Camera image is blurred.”	The remedy is to hold the device steadier
Quality recovers	“Camera clear.”	Tells the user that silence is meaningful again

Announcements are debounced over roughly 0.4 seconds, so a single dropped or motion-blurred frame never makes the assistant cry wolf, and a given condition is announced once rather than repeatedly. A change of condition is announced immediately, because the remedy has changed. This is eleven of the project's thirty-seven passing tests.

6.3 Interaction model

All interaction is spoken output plus a small set of physical keys chosen to be findable by touch. Nothing requires reading the screen or locating a control with a pointer.

Key	Action	Why this design
Space	Read any text in view aloud (Tier B OCR)	Largest, most findable key; the most-used deliberate action
Escape	Stop speaking immediately	Universally understood; must always be available mid-utterance
Arrow up / down	Increase / decrease narration frequency	Lets the user tune verbosity in the moment, without a settings screen
Enter	Describe surroundings now, on demand	For when the user wants a summary rather than waiting for events

Each key produces a short distinct audio confirmation rather than a spoken sentence, so acknowledgement costs a fraction of a second instead of competing with the narration channel.

6.4 Speaking the user's language

India is multilingual, and an assistive device that only speaks English excludes a large share of the people it is meant to serve. That makes localisation an architectural requirement rather than a later feature, so no user-facing string appears anywhere in Drishti's logic. Directions, distance vocabulary, sentence templates and camera messages all live in a phrasebook; the policy engine decides what to say, and the phrasebook decides how it sounds.

English and Hindi phrasebooks both ship today, and a unit test asserts that no English string can leak into a Hindi utterance — which is how the abstraction is kept honest rather than merely claimed. Voice selection matches: the application enumerates the voices installed on the system and picks one matching the configured locale, falling back to the system default with a logged warning rather than failing to speak. Adding a language is a data change plus a system voice, not a code change.

The Hindi wording ships as a first draft and is listed in the risk register as requiring review by a native speaker before the final submission. Machine-plausible translation is not the same as natural spoken phrasing, particularly for terse safety utterances.

6.5 Coexisting with the screen reader

Blind Windows users already run a screen reader — most commonly NVDA, which is free and widely used in India, or JAWS. Those tools own the audio channel, and an assistive application that talks over them makes the machine less usable, not more. Drishti therefore treats the speech channel as shared: it speaks only when it has something that passes the salience threshold, keeps utterances to a few words, and yields the channel immediately on Escape. Verifying correct behaviour alongside NVDA is an explicit item in the user-testing plan below, not an assumption.

6.6 Distribution

Drishti is packaged as a single signed Windows installer targeting Windows on Snapdragon, with models bundled so that first launch works offline. There is no account, no login, no API key and no configuration wizard — because a setup flow that must be navigated visually is a setup flow the target user cannot complete unaided. Because AI Hub's optimisation workflow targets the Snapdragon compute platform family rather than one specific SKU, and because the provider chain degrades gracefully, the same installer is expected to run across Snapdragon-powered HP models, which is what makes an institutional rollout to a school for the blind or an NGO practical rather than theoretical.

6.7 Validation with actual users

The single most common failure in assistive technology is being designed entirely by people who do not need it. Before the final presentation, Drishti will be tested with blind or low-vision users through a local organisation, with three specific goals: tuning the narration policy (utterance frequency, phrasing order, which object classes matter, where the hazard thresholds belong); confirming the Hindi phrasing with native speakers; and verifying that Drishti coexists with NVDA rather than fighting it. The policy exposes every one of those thresholds as a parameter for exactly this reason. Findings, including any that contradict the current design, will be reported in the submission.

This is stated as planned work, not completed work. Nothing in this proposal claims user testing that has not yet happened.

7.1 What has been built and verified so far

The repository is not a plan. The following is implemented, committed and passing today, on ordinary hardware, before any Snapdragon device is in hand.

Component	Status	Evidence
Narration policy engine	Complete, tested	17 unit tests passing
Frame quality gate	Complete, tested	11 unit tests passing
Metric depth calibration	Complete, tested	9 unit tests passing
Localisation (English + Hindi)	Complete, tested	Covered by the policy tests
Provider fallback + context cache	Implemented	Needs Snapdragon device to validate
Model wrappers (detection / depth)	Implemented	Needs exported artifacts to validate
AI Hub export + profile script	Implemented	Needs an AI Hub API token
End-to-end benchmark harness	Implemented	Runs headless on synthetic frames
OCR recognizer decode	Stubbed	Documented as outstanding

The stub and the unvalidated components are listed as such in the repository README rather than glossed over. A reviewer can clone the repository and run all thirty-seven tests in under a minute without a camera, an NPU or an AI Hub account, which is the fastest possible way to establish that the core logic is real.

7.2 Three defects the tests caught

These are included because they are the best available evidence that the testing is real rather than decorative, and because each one changed the design.

Depth jitter re-triggered announcements. The first novelty-suppression design keyed on a coarse distance band. A monocular depth estimate oscillating either side of a band boundary — 1.98 m, 2.03 m, 1.97 m — therefore re-announced the same obstacle several times a second: precisely the chatter the policy exists to prevent. The fix was to key on object identity and bearing instead, and to treat approach as novel only when the object has both crossed into a nearer band and closed at least 25% of its last-announced distance.

A single bad frame moved every distance. The depth calibrator originally used a plain exponential moving average, which is not robust to large outliers: one frame misreading a poster as a person shifted the calibration by 73% and therefore shifted every distance the assistant announced. The fix was explicit outlier rejection — a fit disagreeing with the running estimate by more than $2\times$ is discarded, unless it persists for five consecutive frames, in which case the scene genuinely changed and the estimate re-anchors.

Darkness and occlusion were being conflated. The quality gate's first test fixture for a dim room was not actually dim — it had bright stripes that pulled mean luma to 70 — which hid the fact that the two conditions need different tests and produce different advice. Separating them on the joint condition of low mean luma and low variance is what lets Drishti tell a user to move their hand in one case and turn on a light in the other.

7.3 Demonstration video storyboard

The demonstration is planned as a shot list rather than improvised, because the point of each shot is to make a specific claim checkable on screen.

Time	Shot	What it proves
0:00–0:15	Aeroplane mode toggled on, on camera. Application launched.	The offline claim is shown, not asserted
0:15–0:45	Indoor walk-through: obstacles announced with distance and bearing	Real-time fusion of detection and calibrated depth
0:45–1:05	A hand covers the lens; “Camera is covered.” Hand removed; “Camera clear.”	Degraded input is announced, not silently swallowed
1:05–1:25	Space pressed on a printed sign; text read aloud	Tier B OCR on demand within the same budget
1:25–1:45	Same walk with the locale set to Hindi	Localisation works end to end
1:45–2:00	Terminal: provider line showing QNNExecutionProvider, plus benchmark output	It is genuinely on the NPU, with numbers

7.4 What the final submission will contain

- The demonstration video above, under the platform's time limit.
- The public repository, with setup instructions, the AI Hub compilation and profiling scripts, the full test suite, and the benchmark artifacts as committed JSON.
- A benchmark table giving per-model on-device latency from AI Hub's device farm alongside sustained end-to-end frames per second measured on the target machine, with a same-machine CPU baseline for the speedup figure, and the float-versus-w8a8 accuracy comparison from Section 4.4.
- The architecture diagram in Figure 1, the risk register in Section 8, and a written account of the user-testing findings from Section 6.7.

8. Risk register

Listed because a proposal that identifies no risks has not been examined closely enough.

Risk	Severity	Mitigation
Monocular depth is not metric; calibration may be poor in scenes with no anchor objects	High	Anchor-based least-squares fit with outlier rejection; distances degrade to coarse bands rather than false precision when uncalibrated
Speech output may be experienced as too frequent or too sparse by real users	High	Every threshold is an exposed parameter; user testing in Section 6.7 exists to set them empirically rather than by intuition
Drishti competes with NVDA or JAWS for the audio channel	High	Short utterances, salience gating, immediate yield on Escape; explicitly verified alongside NVDA during user testing
Quantization to w8a8 costs accuracy on small or distant objects	Medium	Float-vs-w8a8 study in Section 4.4 with a decision rule fixed in advance; ample frame budget to fall back to float
Hindi phrasing is a first draft, not native-reviewed	Medium	Flagged openly; native-speaker review before submission; English remains the default until confirmed
YOLOv8-Detection is AGPL-3.0, which propagates to derivative work	Medium	Acceptable for a public research repository; documented openly, with a permissively-licensed detector identified as the swap if commercial use is ever intended

Risk	Severity	Mitigation
End-to-end latency dominated by non-NPU stages (capture, NMS, letterboxing)	Medium	Benchmark harness reports per-stage timings specifically to locate this; roughly 90% of the frame budget is unspent
No physical Snapdragon device in hand during early development	Medium	AI Hub device provisioning gives real on-device measurements without owning hardware; provider chain means the same build runs on the development machine

9. Roadmap

- **Phase 1** Export, quantize and profile all three models through AI Hub device provisioning; record the real on-device numbers, the NPU layer split, and the float-versus-w8a8 accuracy deltas.
- **Phase 2** Wire the exported artifacts into the model wrappers; validate the full loop on a Snapdragon-powered HP PC; complete the OCR recognizer decode; verify the context cache actually removes the cold-start delay.
- **Phase 3** End-to-end benchmarking with the CPU baseline; a sustained thermal and power run to substantiate the all-day battery claim.
- **Phase 4** User testing with blind and low-vision participants; native-speaker review of the Hindi phrasebook; NVDA coexistence check; retune the narration policy from the findings.
- **Phase 5** Installer packaging, demonstration video, documentation, and the mentor presentation.
- **Beyond** Currency-note recognition, medicine-label reading, and a voice-command interface — explicitly future work, not claimed in this submission.

10. Intellectual property and eligibility

- Drishti is a new project, conceived and written solely by the submitter. It shares no code, repository or idea with any team project, hackathon entry or prior work. There is no shared-ownership ambiguity to resolve.
- All three models are obtained from the Qualcomm AI Hub model library under their respective upstream licences: Midas-V2 (MIT), EasyOCR (JaidevAI upstream licence), and YOLOv8-Detection (AGPL-3.0, noted in Section 8 and in the repository README).
- Because the project is newly created for this Challenge, the “significantly modified” requirement that applies to pre-existing proposals does not apply here.
- Single submission; contains no confidential information or trade secrets; written in English; all submitted material is the submitter's own work.

A submission cannot be edited after it is entered on the Challenge Site. This document is the reference draft; text will be trimmed to each intake-form field's limit before entry, and every published figure quoted here will be replaced by the measured value once the AI Hub profiling jobs have run.